Almost. A little thin, but you know. As long as the transcript going okay. Thank you. Yes. Um you want to come in for bite temporal loading? Yes, good. Okay, good morning, everyone. So we have two guest presentations. One on bite temporal loading and one on deployment. Uh can I charge my laptop? Good, it's it's the G system? I can just for that one. Can you help mear a screen? Okay, when it should stop sharing. Okay. Is it a transcript on? Nope. Nope. It is. It is. Got it. It's not sharing. Can take a while to warm up. Oh, that's what it is. HMO four five three is angled. Alright, everyone. Um since the transcript is on, my name is Chris, and if I completely bombed this presentation, then my name is not Chris. Um somebody else. So I'm here to talk about by temporal loading and what is bite temporal loading. Well, basically when you push new data into any table, there's different ways to do it. There's different types of incremental strategies. There's like the common ones that you guys know, which is like truncade reload, and then there's also insert override. And for some of you data engineers, you might even know SEB2. SCD2 is basically a way to track historical data and add effective end and start dates. So for example, if we're talking about associate data, which is what we're presenting on right now, we have a list of associates, and every single day they're giving you the same associates, and sometimes they'll give you a change in the record. So like for example, me, we have tenure month, so in the next by the next month you'll see a new record. Can you count for that percent? Um we will create a historical track of all the data before and then change it. I'll show you guys. It's it's hard to explain. Um but then there's SED two, right? What is bite temporal loading? Bitemporal loading is basically SCD two, but on steroids. So instead of system start time and business, you'll have business and system start time. It'll the presentation will make sense once I show you the data. Um so how do we load bi-temporal load um loading? We we have um it's a simple macro that we have in the prepared layer. So yesterday we did a example we use Mart as the prepared layer, but Mart and prepared are the same thing essentially. So what you can do is you can just simply type incremental strategy equals by temple load V2, and then there's a different load types. There's full, there's um

increment, there's balance. Um, but the main two things that we kind of use is full and increment. And the way to distinguish between the two is whether or not you're getting partial data or full data. So if you're getting full data in and in this associate in table, we are getting full data just put full. But some um data like transaction data, etc. that's coming in as incremental, so what you would put load type equals incremental. Okay, so what is bite temporal loading? It's probably easier to explain when you guys see something. So here's a query that we have. This is the associate dim table that we created using this uh DBT. Um so here I'm querying for myself and I'm doing an order by on system start time. So what is system start time? That's basically the load time that's coming in. So every single day, this job is running. And it's running through airflow, there's an airflow variable that's stamping the system start time. And that's exactly what this is. So if you scroll to the right, for example, and this is a small screen. But if you scroll to the right, I've ordered by the system start time, and you can see that it has kept a historical record for me in the entirety of when this table has been created. So it was started on the so we've actually loaded historical data here. So you can see that the system uh is laser, but let's start from here. So this table was deployed in the Caspian on the 26th of 7. Well, actually. It started, it started the new updates on uh 07 17, exactly. Okay, essentially. So since then there have been multiple changes in my name. Uh like one, two, three, four, five, six changes. And you can see what those changes are. So it's tracking everything, really. Whenever there's a change in a record, like for example, what happened between this row and this row. The tenure month, so I've been one month older in CG, and that's what's changed. If you look at the next few rows, you it's actually funny because there was a bug in our code, which I'm not ashamed to show you, but there is something here, call it right here. At first a job code and job title wasn't showing up properly. Um so like as a data engineer, you're looking at the code and you're saying, is that coming from upstream? Is that coming from us? In this case, it was coming from us. We weren't doing the join properly. So you can see here that I want to highlight this because you'll see tenure code is still 53 here, but there's a

new record, right? So the new record is because of that bug that we fixed. So now it's like, oh, now it's properly showing up the job code and it's properly showing up the job title, and so it sees it as a new record, so it tracks it as a new record. And then you can see that it's resumed back to normal with 54. So I mean that's essentially what bite temporal loading. Oh, forgot to show you about the DL columns. So to the right, there's a lot of BL columns, and this is kind of what bite temporal loading is. There's business start time and there's system start time. So system start time is basically when the thing is processing, that is the exact time. So it's passes at 6.42 am at this specific day. In this time zone. In this yes, I actually don't have these time zones memorized. I always have to reload. I think it's UTC, if I'm not mistaken. It might be, yes. It's UTC, the TTC. And then it has a start date, business start date. So it's essentially just the for the most part, it'll probably be the same. Um it's very rare that these are different. Uh I can't really think of an example off the top of my head of why this would be different, but you can see. Well, actually, this is different right here. I have to look at that. Sorry. Are the business start and end dates part of the DBT part of your code, or is that inherent in the platform from the like parting of the system group? It's part of the macro. So the way the business start time works is it's taking the system start. Well, actually, I actually scoped that too, to be honest. Because I felt like that was going to be a question that was going to be asked. It's in this macro here. If you could see if you see start date column, it's essentially load timestamp minus one millisecond. That's that's what the business uh that's the logic for business start time. So in this example, why did that show up as seven? It's data for nine scene in for 1948. It starts. That's why I screamed too. I think there's there's weird things with um time zones here. So this is UTC, but it technically ran on PST, which is technically an earlier time than this. So that's I'm assuming that's why business start time is a day earlier in this example, or in these examples actually. Yeah, and the reason I bring it up is that as a data creator and a data owner, the navigation and mental model of understanding by temporal, bite temporal loading variable, and business effective dates that are called out in the data itself that this

is the date. Um a lot of data engineers and technologists historically have just used system time tracking dates just for eons, as oh, this is when that happened, and this is when it's effective. Um really makes that confusing for a lot, a lot of people. So it's imperative that when we model and when we talk about our data products, we're talking about dates in the proper way. Um ideally with effective dating, and then also ideally not with uh Boolean flags, or at least carefully considering the use of Boolean flags because they for status things just say on off, uh, is once you start looking at bite temporal and business effectivity, those Boolean flags lose meaning really, really quickly when you're looking at things across snapshots and across the different bite temporal states. So just no really think about that. No, Jeff, you actually bring up a great point, and as you were speaking, a light bulb clicked in my head. And so system start time, just so I can close a loop on this question here. System start time is when the time runs. Business start time is the day that that the active day for that and in terms of business reporting. So for example, we run this every morning at like around 3 a.m. PST. But the data that we're running this for is for the day before. That's why the business start date is 27th, because this is as of 27th. But technically it was ran on the 28th. So, in terms of like what Jeff was hitting on the money here with like business reporting as a reporting or like as over as was or etc. A lot of the times when business need to query on specific snapshots, they would want and they want to see at that business day, they would use business start time. If they want to see when it actually loaded, they would use system start time. And furthermore, you can also have something called the current indicator if you just want to see what's current and currently available for everyone. That's when these two dates come to pay and I think the business date can change with the system date. Bitemporal allows for corrections, those are manual stuff that we would have to do. No acid date connections from the full system, right? So if I oh right, right, right. Yeah, yeah, yeah. There's an active two, but that that's something else. Yeah. There's an inactive record. I don't really want to do this active. So the main purpose of the bite temporal is to have the capability of S is

and S was. So we can go back and pass and see how the record looked in a certain point of time. And that's what this system dates are calculated. System date and business queuing date. Right. And thank you for sharing the bug and the defect, right? Because I think that's a great example. It is, yeah. Right? And it's like, well, that's not expected. Well, yeah, it's just it's a it's a true audit of what was in there at that based upon the code that executed at that time. Yeah. Yeah. And what's this mark right now? I mean, I tried to understand this like the DBT feature. Or DBK just the engine which execute and you're old the loader within the mark code. Yeah, so everyone's unique by the old role, right? And you're needed for the neural as in the company problems probably and they like doing it with like a chip sound or uh is your question how is it working? Yeah, my question is a logical bite temporal things uh defined in the micro or is it something which is supported by the DBT? You know, I I have the same question, so I asked AI that just this morning. Um so it's actually defined in macro form. So we have Ginja templates that are referring to other ginger templates. It's just super abstracted, really. And then eventually there is Python code that's doing all that. So this is this is the the source code of how uh our how we've programmed the bi temporal loading. So I mean we you can go through this. I haven't really fully gone through this, but this is how essentially we are doing like we're stamping, we're we're creating uh SHA code for each new change, and then we're stamping that, and we're we're just that's how it's working. But it's uh it's all abstracted through Ginger and macros. It's just more like not all the stuff here, yeah. You can actually see this. I mean, yeah, I can add a little bit more between the what is the incremental load versus the balance uh full. Yeah, what we're balanced. So there are eight strategies, I believe, on the yeah, we do we you know so far. I some data engineer I've worked with these three. Um I don't think I build any balance table yet, but I can talk technically about the two. So the main difference between full is when you get the source file that is sending you a full file, for example, the full, you know, full file, let's say book of record or accounts. You will get a full um uh full file with the whole book of record for accounts every day, for example. In

that case, you can uh use uh the full. Usually that's where we use the full uh balance strategy. Again, as a data engineer, you know, you need to consider the cost and how big the data sets are because the pool running full on the daily, it's very expensive. Uh because you are really end dating every single record that came previous day and inserting a new version of that record, uh, you know, again uh today uh that will have the DL current indicator. Uh and then uh incremental is mainly used in my use cases, it's mainly for the financial transactions, for example. That's a good example where we are getting, you know, every day there is a lot of the transactions, and you are never gonna get a full history of all the transactions every day. So you're getting incremental files, and in that case, we are going and loading uh, you know, when the new file arrives, those are usually gonna be insert, but um, you know, uh what incremental is doing is you know it's gonna go and check, you will determine the the the you know the business key is very important, let's say transaction ID is your business key. So you can go and see if the transactions usually have corrections sometimes from the source. So the system will go and it will compare based on the transaction ID, see if this transaction ID exists, it's gonna end date that previous version that exists in the system, uh, end date it with a DL system uh, you know, the DL columns, like business depends which strategy, but it's gonna be you know either business dates, um system dates, and it's gonna insert a brand new version. So what this does, it's only gonna touch the records that show up again. So if it doesn't find a transaction ID, that's a new insert. So it's handling your insert. If there is an update, it's gonna go and end the previous version of the record, insert a new one. And that's exactly the capabilities that you're gonna get because you can go back in time and see, hey, why my uh Chris got credit for my transaction. Well, there was like an issue with you know the transactions came different with the wrong app ID. We got the correction, it has a different rep ID, and now I get credit. So you can answer those historical questions. So how often does a new transaction come in? Or how often? The new transactions come every day, for example, but the corrections it can happen

anytime, right? So let's say with CDM data, the fading identities, for example, is a good example. We can keep the track of the changes that are happening because we're I'm managing the transaction for dealer branch office today, for example, and it's gonna match to the my rep 123 today, because that's what my trading identity uh gave me. But tomorrow they changed, hosting made some change in the you know on CDM data, and I have a brand new trading identity, I can keep those changes now and answer or you know, uh answer the historical questions. So it's very powerful, and you don't have to worry, you just need to understand how the data is coming and determine what is the right strategy to use, and then the framework we have on Caspian, it's gonna take care of multiple. Obviously, there are so many different things to consider when you're modeling because nothing is black and white, and there are different edge cases that needs to be handled, and yeah, framework is not gonna work for every single case scenario. Just this year I had two projects that didn't fit in any of these models, so we had to build the custom things and plot and couldn't like prioritize the changes I requested and so on, but uh you know that that's the main difference. The the key point is uh by temporal helps you answer S is S was. Any questions from anyone? Balance, I I only know balance is mainly used when you have the dollar amounts and when you're you know for the financial data sets. That's most commonly used. I haven't used it personally because we didn't have a use case. Yeah, and just summarized uh how it's working, DBT does not provide a um by temporal strategy. So you have to build your own. Yes. Which is essentially what this is, what I'm showing right now. I think the gentleman over there asked how it's working. These are all the macros. So if you look at this macro, you can just keep following down this lineage of how which macro is calling which macro, and eventually you get to this pipeline script. So if you're curious about that. And when you guys vend your um your product, it'll automatically have these by temple stuff for you. And if you need to make changes or custom stuff, then you could technically do that too. Okay. Why don't we get why don't you get yours after lunch? Just create a bit of a kid. Um what

we'll do now is we'll go through aircraft a little bit. Um is and then just have a set variety delivery and just like the format. Okay, cool. Um right, so uh you generated some airflow yesterday with for the day contract, right? Um let's talk about airflow a little bit. And then we'll give you the app we're trying to do it today to work through. We'll walk through them similar to how we work through the DBT stuff. Um the fade this phase one primary phase, phase two with a flow, so it's simpler, shorter. Um at the when we've done airflow, maybe we'll be done with a flow potentially by let's say 130 or something, right? Then if that's true, if it's true, we can talk about CI CD in deployment context around uh deck products. It's not relevant, not directly relevant to their product creators as a practice that helps understand how the system RAM products works. We talk about that bit a bit. Um and then we can show off some a variety of tools. Maybe people have access actually to data dog and harness and other systems, but even Amazon S3 to show some lambda functions being triggered. If we don't, if we do, then we can show that. Um if we don't, then I can show some screenshots and stuff. Cool. Then on Friday we have a capture and event. Um there's caption comes in two phases. Phase one is a sort of minimum certification phase. That takes like a two, three hours or something. I don't know. Uh once you've done phase one, you kind of set a five TPC. If you finish phase one, there is a phase two, which has lots of open and distract challenges. Uh if some people look at finish phase one quite quickly, uh I've got another capstone, which I might ask you to have a look at uh rather than the phase two, if you're quite advanced. Um the way that the capstone on Friday is structured, is it's uh kind of it's um simulated data set rather than a CG data set. And the one I've created, I've created another one, which is a CG text set. But um, some very advanced people in the room who get finished with the non-CG one. Uh I might ask you to have the CG ones to see if it's any good. Cool. FO. What is Airfo? No, okay, code for me. Oh no. So orchestration. Uh which means the starting and where does orchestration come from? It comes from orchestra, right? Roughly. What is how does orchestras work? Well, they're timed by conductors, right? So use how it's all. So that's the

idea of the orchestration. So it's fighting to be different parts of the orchestra. Right? And you're getting things to work to work together in constant, right? Literally called in constant working together. Okay. Let's look at some Airflow code and things, I guess. So uh so when we're dealing with it with Airflow, um, we're typically writing Python function, right? So if I say def my task or something, my task. Now the structure of this function, generally speaking, in the first part of the function, we have what's going to call operators. The end of the function, we have some syntax which describes how the tasks we define here are sequenced. If I say task one and then task two and then task three. Cool. Have we seen Python code before? Yes. Blank expressions. A blank expression is neither a yes nor a no. So if you if you want to know how the one interpret a blank expression, so maybe you're just maybe just know all this, and therefore you think blank means I'm going to interpret the length as yes. Okay, so in the operator section, this is where we connect airflow to some alternative system, right? Now the dbt task group comes from a package called Cosmos. Cosmos or Cosmos. Cosmos is an adapter between Airflow and DBT. Now, why are there all these words here? Because I don't know, tech people get the word when you name things silly, right? So if I was naming this, I call it Airflow DBT adapter. But yeah, I'll be nice with name. Now you want clock models, right? You want some whiteboard somewhere and then some product managers be confused and go, what's called to go, ah yes, that's the airflow dbt adaptable. Now anyway. So rather than doing that, I would uh anyway. The point is the dbchide task group operator comes from the cosmos package. There are other operators in Airflow. You can have an operator that, for example, connects Google Docs or Slack or Teams or other things to Airflow somehow, right? So all these packages that can give you airflow tasks. Any case, a DBT task group is basically a operator that that picks up a can that picks up configured DBT pipeline with all the YAML files and everything, and just represents each stage of the pipeline as the as a airflow task. If you look at the airflow UI, you see all the steps in green and yellow and all orange and stuff. Each of those steps is a is a like for example stage of the

intermediate presentation. Okay, now the last part here is the kind of interesting part. We've got we've got T1 and then the double double gray and some symbols, and I read those as and then. So this is the sequencing operation. I'll I'll replay it here. If I say task one and then task two, and then I can even say, for example, list task three, comma, task four. Cool. Now if I do that, what's happening there is I'm getting task one run, and if that succeeds, and then I I follow task two, and if that succeeds, I go to three and four at the same time. Now, um if a data this now, this this whole thing here is the DAC. That's the DAG. That's what we mean by that. Cool. Uh that's a directly basic graph. Now, um airflow has some options. So if I if I create if I create a function if I say def in my task, right? Above the task, sometimes we'll have the app symbol. And we'll have DAG on those lines, and we can configure Airflow. You can configure Airflow to do different things depending on the behavior class. So for example, the thing called callbacks, the success callback, failure callback, SLA callback. Now the reason we have um those configured as part of the airflow configuration rather than as part of the DAG itself, is because what happens when the DAG itself fails. So we don't we don't, you know, okay, how do you handle failure as part of something which is self as fail? Does that make sense? So you have like an escape hatch. So the escape patches call an on-failure callback. There's also on success callback is on SLA failure callback. Well, callback is it's another function that is run when an event happens. So another piece of code runs when this event happens. So when, for example, failure happens on a DAG, what the failure callback can do is, for example, log things did the data or something locked, lock, lock or something like that in the right. Now obviously I'm just sketching here, but you have challenges to go through which your code is sketching. Okay. Um the airflow UI, it's what the FO UI for the moment. The FO UI, you've got two parts. You've got your DAGs here, which are your task sequences, and you see you know, green, amber, red, as your task, you know, your your phase status and things. Then on the right-hand side, you've got like you can see the code, you can see various configuration options on the right hand side, you can press code, you can see a

code time tag is nice. You'll see all this in the UI. Just highlighting. Now, uh final thing, I guess. Astro. So that's Airflow. So in Airflow, we write uh DAGs in Python. The main operator, the main operation we're doing is an and then, which sequences tasks together. And the main thing we're doing is we're bringing in operators like dbt task group or database operator for a notebook or something. And um, that's it. So actually, a lot of our airflow code can be quite simple. It can just be like here's a database operation, here's a dbt operation, here's some other operation, and then your sequence those with this symbol here, right? Now, of course, in practice, all this code can be generated for you, if you have any need to do it. But in practice in practice, we're using Astro. The Astro has blueprints. Blueprints. The blueprints are cases where people have already written the F word code in Python, and the placeholders in that code, and all you need to do in the YAML is fill out those placeholders. Make sense? Now, for probably a very large number of day products, I guess 90% plus, um, you can just use the blueprints. And if you do have a custom airflow case, probably just create your own blueprint, and then that becomes another blueprint for the organization. Right? Makes sense. There's a number of different kinds of scheduling tasks or things that are really happening is there's probably low variety though, low variance. Does that make sense? Okay. Um the challenges are all around the Python side, because once you've got the Python side down, the actual YAML stuff is very simple. It's just a very simple configuration. Any questions? No? Okay. Uh the challenges I believe are O three. Some of them are a little harder than D Bt, I guess. Um there's some data log examples there of like some interesting things. Um we'll go to 130. Now at 130 we'll talk about CICP and we'll talk about the deployment context and things. And uh cool. All right. And again, if you notice any issues with the challenges, feel free to send me a message on Teams or something. Uh as I said, I don't own the repository. Uh yeah, but you don't do, so get Martin for change or something. I think it should be good. I think it should be good with now in the actual CLIST and the same thing. Um Michael. Yes. Can I ask you? Well, you can see something.

Good afternoon, everyone. Hello. Any people have done phase one? Okay. Let's talk about CI CD briefly and draw the broader deployment context of data product. And then we've got a guest representation. Adds some even more. So that just define CICD to what are we going to talk about and then okay? So what is CICD? What is deployment? Continuous integration continuous deployment. Good. What does that mean? You're always building and then always deploying code. Yeah, so the thing we're integrating is what? The code through the SDLC. Okay, good. So integration, I mean what integration basically means is the word merge, git merge. Right? It means combining one feature from a branch. And continuous means what? Frequently, yeah, frequently. How often is frequently? Under man slash. Yeah, good yeah, yeah, yeah. Uh like daily with the hourly, and whatever, right? Not monthly, and not yellow. Yeah. Uh deployment means what? Deploying a big release of software to where? Up in environment or up a room. Production environment, yeah. So uh so continuous release production, continuous merging of new features. Why do we do CI C D's why we do CICD? I'll probably data products in a second. Why do we do CI C D? What's the pointer? Automate? Automate. Ship. Sorry, ship, yeah, it is automation. So what's the point of automatic? Why bother automating it? Yeah. Production? Yeah. Yep. And what's the value of that? We're not doing anything. You're not saying one. We are not breaking anything. Bricking anything. Yeah, so we're we're get we get in we get faster feedback in the development cycle of things being incorrect. Right? So think about the beginning of a of a of a product and the end of a product. When do we want information about the product being broken? You want it as fast as possible, right? So we can iterate on it, yeah. Now that's one value of CICD. The other is that when we're working in Teams of people and there's a possible code drift, like a product drift, we're even called it, um, by forcing integration often in the deployment, is a team has a common shared understanding of what actually your product. Right? So if you set a team off for a month just to do something, they want integrating frequently, then people wouldn't have a common understanding of what's going on. Make sense? So you want to team to have a

shared understanding, and you want um the surface errors and issues quickly, or that's about adding kind of feedback and observability into the development lifecycle, yeah? Okay, good. Um CI CD. What's we do downsides? People tell me not to do limitations of things, because I'll get in trouble. But I'm gonna do many. What's well what could be a downside of CI CD? Is there much of a downside? Button? I mean digital. Yeah, so we got uh when we start automating things, we can have issues with automation, so we can have lack of human in the loop. Or the human in the loop is just a board person going yes, yes, yes to everything, right? So that's just might as well not have not have a human in the loop, right? Um so the more you automate sentence, the more that they're fragile to some degree, yeah. And more you cause correlated failures, you know what correlation in markets, right? We don't want correlated returns, do we? We don't want the market to go, you don't want all of our fun, yeah, all of our portfolio to go up and down at the same time. We want some kind of mixed elements and portfolio, so we don't know those all our money ones, right? Well same for deployment processes. We wouldn't want all our deployment processes to be correlated, right? Because if the one thing went down, the entire system would go down, right? That's why when you see, for example, Google depo down on Amazon go down, we don't live in a world anymore where GitHub goes down. We live in a world in which if GitHub is down, half the internet must be down. Right? Because all of the systems are correlated to add to Amazon, East Coast, data centers and stuff, right? So massive amounts of technical correlation. So when one system is down, probably you can't do anything. Okay. Good. So the ratio of correlation. Also, when you once you have automation, you then have people maintaining the automated thing, right? If you just have to just have people releasing software themselves every month or something, they would you know type a few lines of code or whatever and they press play, and then that would be the release. If you have an automated system, you then have accountability auditing all this. We have to have a proval system, you put all this other, right? So that's that has a maintenance burden. Anyway, so it's not free. It's not free, CICD is not free.

Okay, good. So um what do we use? So what so when we say deployment, where are we deploying things? So we've got data products here, right? So data products in this world, they're DBT in their vote. So that means their code in a git repository. Yeah. Uh unlike, for example, notebooks in databricks, they're code in a git repository. Uh so they're code, and we're versioning them with git. Where does that um where do where are we deploying that to? So dbt goes somewhere, airflow goes somewhere. What does DBT go? Data bricks, yeah, where does the airflow go? Okay. Um, now there's a um there's a CI C D system at uh CG called harness, talking about harness. So harness, you can people wrong get presentation harness as well a little bit, is harnessed is approximately approximately a GitOps CI CD system. What that means is you take CI C D, which remembers continuous integration and continuous deployment, and you make the trigger for deployment, you make trigger for integration, a GitHub event. Right? So rather than having a button that someone says play, we're now in deployment mode, and like those often screening, right? Um, you make it a PR request. Go to GitHub, someone does a request and merge code, someone approves that request, and then that triggers your release process. Oh. Now why is that valuable? That's valuable because, first of all, from your point of view, it isolates you from build the bigger deployment machinery. All you need to do as a data creator, for example, is have a GitHub repository, have a pull request, submit your pull request in GitHub to say I've finished my data products and approves that, and then in the background things to go on. So get GitHub effectively kind of black boxes or isolates you from the broader deployment system. The other value or virtue is that your git history, your version control history in Git becomes a history of your live environment idea. Or at least your development. Make sense? The left eye is bored, the right eyes confused. The mouse is in Santa Fe. Dreaming about some green. Okay. So now you can push this further. You can go away with GitOps. You can literally make Git repositories through an intermediate piece in a piece of software. You can literally make git repositories mirror the history of your live environment. You could have configuration files in your gate

repository that describes a live environment. And whenever you change another configuration file, a live environment changes. So you can browse the repository as if it were just a record of what's running in your system. Cool. That's kind of a modern way of doing CIC. So exactly what you have. This is more like triggering things based on rather than the get literally. Okay. Time for presentation. I don't know what the presentation is. I just said can someone show me some harness stuff and deployment stuff just so everyone gets a chance to look at it. Yeah, I can show what we're doing. Yep, okay. It might be, it might not be. Whoops. You should just be able to plug something in. You can unplug this. Okay. Yes. Oh, you usually come on when you plug something in, that will say. Surely no. No, no, yeah. Okay. Okay. So this is the harness user interface. Usually you save the link, so just go in. So I have multiple um environments. Let's take with a Caspian. So in Caspian, how we deploy the code. You know, here you can pick which, you know, I will have laser and Caspian. So you pick which one you want, then you can go to a pipelines. And this is where we usually search for repository. You know, our Git repository that we want to deploy for. And then here you will see a multiple environments. Let's pretend we want to push something in the SQA to test. So ideally you will have a feature branch that you want to deploy from. And maybe you don't want to do any PRs at this time, you just want to test, you know, your deployment. So you will come here, click the run button, it will pop up a window. Uh and then one thing you have to do here is because this is SQA, we only have one parameter. So you will apply this input set. And here, um give it a second. You will have the path of your feature branch, or you can search for it, feature, and select, you know, wherever, you know, this is something old, so I'm not gonna release it. And once you select your feature branch, you need to make sure it's correct feature branch. Where your changes are, you can uh run the pipeline and it will then walk you through the steps and deploy. Usually it takes a couple of minutes. But one thing before you even go here, when we go back to pipelines. You want to make sure, because this is a shared environment, most cases, is uh you want to check who did the last change. And in this case, we see

you know, the initials of the person who did the change will be there, so you want to check with them, hey, are you done? Because when you push your changes, you may override their changes. And if they are in the middle of the testing, their testing fails. So make sure you know those are some things that we usually check for. Um when we do it when we do the true harness. So when doing the production release, obviously you will go to the PRD one. This one is for the uh production. So not everyone should be doing this. That's the first step. Uh again, you will click on the run. There is just additional step. You know, here uh when we do the you know, uh we will do, we need to select fixed value and do um PRD. Uh I think we're doing the PRD build inputs again. I'm not the one for the releases, so but I believe that's the case. So again, um, you know, here it will be you know, the same window pops up. You you know you will have to do again fixed value. Either you have your release branch um with the name and the date. So you put that here. But one important step that needs to happen before every release, what do we need? CRQ. So CRQ, you go here again, fixed value, and then you put your CRQ uh number. One, two, three. So no nothing should go into production without CRQ, and once you post this things, you run your pipeline and it will deploy overlash. So that's all I know about harness. Yeah, there could be usually there are no failures, so there they are. Um, you know, um, I'm not usually the one debugging, I'm just complaining to someone, and then they help me debug, so I don't know how to show you that. Um but you can make more titans and the one do you have an icon here? Like green and those alert modes. Yes, yes, you do, yeah. You there are probably logs and there are logs that you can see. Um so yeah. You can do let's see one of the C on the files, because maybe I can delete others. Let's see here we had a failure. Okay, this was a bad thing. Yeah, I think it's a CLQ model. Well, this is SQA, so you don't need uh string up on the top, like invalid questions the online. Oh okay. Okay, that's what we should thank you. Um yeah, that's that's what I know about harness. I don't know what else. I don't know what else. Any other questions? Anyone else want to say something from the content? Yeah. Anyone who is doing a bit more or from different perspective?

You can get a API token with a scopilet to trigger deploy it for you. Ooh, dangerous. Cool. So we've got we now have like a catalyst for harness, I guess, is some some skills. Well, you can just get an API token within Right. Store that in an in our in variable or something. Yeah. There you go. So if you're working on platforms and release. Any other questions or observations about that? That's a bit different. So we'd also have harnesser CDL. But typically all of our jobs are triggered via website if you want to have redo it. Um what's this one? Webhook meaning Oh like just like I'll say like, oh, I just merge like a future branch to like development. Yep. Yep, it's cool. Let's look through that. So we don't have to like make sure any variables besides like you know, building the actual pipeline. I think I'll be true of most data creators as well. Like if you're creating like a it'll be a standard thing. You can see the like the maybe that's what we guess. Is that like MSS dev? Well why you do that? Are secrets stored in Pinus? Or is there like another system that CG uses? Uh trigger triggering. Yeah, but triggers at the top. Like the very top. Uh right hand side. Yeah, so that's your that's your that's how it's written. Any other product? Any other observations about CIC? They're all the same, actually. Yeah. Yeah. Or not probably all have like stages. They just have steps. Mm-hmm. Cool. Doesn't need to be a lot of depth to it? I mean it's not, I mean it's just background information for cool. Okay. Okay. This is yeah. Alright. Um I don't know. I mean, uh, who's on phase two? Who's finished phase two with getting to phase on phase two? Who's finished phase two? How long does it take to do phase two? Two minutes? Yeah, I think it's a good thing. It's quite quick, yeah. So if my people are on phase two, I don't know. Let's let's see what we get to by 230 then. Cool. Yeah, I think it's um do you know how to bring yourself an explosive? If you do, I would suggest you do that. And then two thirds you can see. And that solves probably more than one. Um what's the problem? But when the platform didn't work? It's just now running container. Did you do phase one? Okay, phase one. That's fine. Alright. Um you can try continue to try and delete delete all the containers. Yeah, you need you need to have a working container for if you want to just delete all the dev lock containers manually and then

depend on you're already trying to. I might have to do it again. Uh we can probably get someone from platforms to come and support. Let me try. One more thing. Just one more thing. Okay, I'll check back in at 2.30. Hopefully you've done phase two. If not, that's fine. But keep going. If you are done phase two, then take a break, do some work. Oh Michael, I think there might have been a question. Yeah, that was answered. The answer is nine. Hey, Michael. So what's uh caps on tomorrow? Uh yeah, so it's you know, it depends when people get finished, but it starts at nine. The sharper you start, the sharper you finish. Um sharply at nine. And then what? So there's like an hour where you are in uh 45 minutes now, whatever, half an hour, 45 minutes an hour. Just on your tables, we can rearrange tables or something. I don't know. Just on your tables. Working backwards from the case study. So the case study is AI generated based on an alternative well fund management company with a slightly different definition of like portfolio and fund. And that's semi-deliberate, which is that part of the opening is to give you a chance to design stuff and work through like the requirements and all that sort of stuff, the PRD, the contract. And if it was just a standard CG example, then it might just be a bit trivial. However, I have gone and created a version two based on CG products. So let's put that aside for now. Um so doing that for like half an hour, you're on the same page, you all you know what you're trying to create, what and then basically if about the you know, it takes you as long as it takes you. There's two phases, there's phase one. Phase one is what you have to complete. That's like the certified creator. Phase two stretch. Okay. Um it typically takes people a few hours, three hours, two, three hours to create to finish phase one. So from like 10 to 1, 10 to 2 is when it gets is when people finish. So if people are finished phase one by let's say 145, suppose that were the case, then we would have 50 minutes of like reporting back. And we will figure out what we can do for reporting back. And then we could potentially be done by two. Um what I would say is for the last 50 minutes, we want uh someone from every group, um, maybe to talk about um we'll give you some questions to discuss, maybe something like um what was the hardest part

of the initial conversation? What was the most confusing or difficult step as part of the project? Questions of that kind. And then um then we can all go. Cool. Sounds great. And if anyone is feeling adventurous, you know, someone who's very advanced who's found all of this very easy. That'll be useful.